Akismet comment-check: API key and Authorization data on HTTP fallback

Technical report | Live sandbox: https://mikesmid.nl/wordpress/ | Tested 22 October 2026 | Akismet package in repository: 5.7.2

Executive summary

A controlled test on the live sandbox confirmed that Akismet adds its configured API key to the body of a comment-check request. It also copied a synthetic HTTP_AUTHORIZATION value from the request environment into the same body. After two deliberately simulated HTTPS failures, Akismet made a real HTTP request to its comment-check URL. WordPress reported HTTP status 200. Thus the fallback is reachable on this installation, and outbound HTTP was not blocked before the WordPress HTTP client returned.

Security consequence: if a real HTTPS failure triggers this path, the Akismet API key and any copied authorization value can be placed in a request sent to an HTTP URL. A real application password was not used in this test. No natural HTTPS failure or historical disclosure was observed.

Key evidence

Measurement	Observed result
Configured key length	12 characters
Configured key suffix	**e16f**
SHA-256 of configured key	cd23f08efb50b582ac3795d732a0df26ec778fa3a3cd921bf3a8b181b45d60f1
Comparison with outgoing body	api_key matched the configured key in all three attempted requests; its SHA-256 matched the fingerprint above each time.
Authorization comparison	The synthetic HTTP_AUTHORIZATION value matched the value in all three request bodies.

The full key is omitted from this report. The site owner can independently hash the configured key and compare the fingerprint. The test output exposed no full credential.

Controlled test method and result

A temporary WordPress MU-plugin assembled a synthetic comment through Akismet::auto_check_comment(). At the akismet_request_args filter, it captured the resulting comment data and stopped before creating a comment or sending it. The captured data was then passed to Akismet::http_post() using the real configured API key. A pre_http_request filter returned a simulated error for each HTTPS attempt and allowed the HTTP attempt to proceed normally. The probe inspected request bodies in memory and recorded only comparisons and a key fingerprint; it did not log credential values.

Attempt	Transport	Configured key in body	Synthetic auth in body	Network handling
1	HTTPS	Yes	Yes	Simulated failure
2	HTTPS	Yes	Yes	Simulated failure
3	HTTP	Yes	Yes	Real WordPress request; final status 200

Root cause in code

1. Akismet copies string values from $_POST and almost every HTTP_* server variable, excluding cookies but not HTTP_AUTHORIZATION. See class.akismet.php around lines 492–515. WordPress can use HTTP_AUTHORIZATION for Basic authentication with application passwords; see wp-includes/load.php around line 106.

2. Akismet::http_post() adds the configured API key to the form-encoded request body via add_query_arg('api_key', ...). See class.akismet.php around lines 1799–1804.

3. The function retries HTTPS once on error, then calls wp_remote_post() with an http:// URL. If that succeeds, it records a 24-hour akismet_ssl_disabled setting. See class.akismet.php around lines 1823–1885. The test forced the failure condition; it did not observe a spontaneous TLS failure.

Scope and limits

The HTTP 200 result is the final status observed by WordPress for an attempt made to an HTTP URL. The probe did not record redirect hops or a packet capture, so it cannot establish which intermediary received the plaintext body or whether the final Akismet response followed a redirect. It does establish that the WordPress HTTP layer accepted the HTTP attempt with the API key and synthetic authorization value present in the body. No real application password was transmitted, and no comment was inserted.

The site’s inbound HTTPS setting is separate from Akismet’s outbound transport. A site served over HTTPS can still take this outbound HTTP fallback. Conversely, if PHP lacks SSL support, the code may start with HTTP without the two-error sequence.

Recommended remediation

Remove the plaintext HTTP fallback: return an error after HTTPS retries rather than retrying over HTTP, and avoid persisting a setting that disables HTTPS. Restrict Akismet’s comment-check payload to fields required for classification; explicitly exclude HTTP_AUTHORIZATION and unrelated POST values. Add regression tests that assert the key and authorization data cannot be sent to an HTTP URL after simulated TLS failures. On a site using application passwords, review whether any such credentials were used to submit comments; this test does not establish historical exposure.

Test cleanup and reproducibility

The temporary probe was deployed in commit 3d5145f and deleted in commit 49c37a8. The removal deploy completed successfully. The probe route subsequently returned ordinary site HTML, the temporary PHP path returned 404, and the homepage and post page returned 200. The probe captured and restored the Akismet SSL and alert options in a finally block. No test comment was created.

Evidence and code: temporary probe source | probe deploy | cleanup deploy

Akismet source reviewed: class.akismet.php | WordPress Authorization handling